

BSc (Hons) in **Computer Science**

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026

Faculty of Computing

Practical 07

> **Module Objective:** In previous weeks, your Drow Ranger agent used A* Search to find a physical path to the goal. However, it blindly assumed every non-wall tile was safe. Today, we implement a **Knowledge Base (KB)** and a **Forward Chaining Inference Engine**. Your agent must now mathematically prove a cell is logically *Feasible* before taking a step, using real-time rules about map vision and enemy threats.

> **Before you start:** This lab continues the project from Labs 1–4. Open the same project and check that `SearchAgent.astar_search()` in `agent.py` works.
>
> - Parts 1, 2 & 4: new files (`logic_engine.py` , `test_logic.py`) in the project folder.
> - Part 3: edit your existing `agent.py` .
> - **Create a separate branch `lab-07`** and start your work there. When finished, **commit your code and push the branch**, as you did in Labs 1–4.
> - **Upload this worksheet's answered PDF to the submission portal.**

Part 1: The Logic Engine Architecture

Step 1.1: Building the Knowledge Base (KB)

> Instead of hardcoding hundreds of nested `if-else` statements in our agent's movement logic, we will build a declarative Knowledge Base. You need to design a Python class that stores **Facts** (current percepts) and **Rules** (Horn Clauses).

1. Create a new file named `logic_engine.py` .
2. Implement a `KnowledgeBase` class based on the structural requirements.

```
CLASS KnowledgeBase:
   // Attributes
   DECLARE facts AS Set                // To store unique string facts (e.g., "TargetVi
   DECLARE rules AS List               // To store rules as Tuples: ( [list_of_premises

   // Methods
```

```
    FUNCTION tell_fact(fact_string):
        ADD fact_string TO facts

    FUNCTION tell_rule(premise_list, conclusion_string):
        APPEND Tuple(premise_list, conclusion_string) TO rules

    FUNCTION clear_facts():
        EMPTY the facts Set
```

Part 2: Implementing Forward Chaining

Step 2.1: The Inference Algorithm

> The agent needs an Inference Engine to deduce new information from its raw sensors. Translate the following Data-Driven **Forward Chaining** algorithm into a Python method inside your `KnowledgeBase` class.

1. Write the `forward_chain()` method.
2. Ensure you use a `while` loop that continues to iterate over the rules until a full pass results in absolutely no new facts being deduced.

```
FUNCTION forward_chain():
   DECLARE new_facts_added = TRUE

   WHILE new_facts_added == TRUE:
       new_facts_added = FALSE

       FOR EACH (premises, conclusion) IN rules:
           IF conclusion IS NOT IN facts:

               // Modus Ponens Check
               IF ALL premises ARE IN facts:
                   ADD conclusion TO facts
                   new_facts_added = TRUE
```

Part 3: Hooking Logic into the Grid Game

Step 3.1: Defining the Game Constraints

> Now we translate the game's safety constraints into our logic engine using Propositional Logic.

1. In `agent.py`, add `from logic_engine import KnowledgeBase`, then create the KB in `SearchAgent.__init__`: `self.kb = KnowledgeBase()`.
2. Use your `tell_rule` method to define the following safety rules:
   Rule 1: `TargetVisible` ∧ `HasDust` ⇒ `SafeToEngage`
   Rule 2: `SafeToEngage` ∧ `BloodseekerMissing` ⇒ `Retreat`

The game does not produce these facts, so you supply them per tile as a dictionary: `{(x, y): [facts]}`. A tile where `Retreat` is deduced is unsafe to enter.

Step 3.2: Validating Feasibility in A*

> Modify your existing A* pathfinding algorithm to consult the Knowledge Base before expanding a node.

1. Add a parameter `tile_facts=None` to `astar_search()` (treat `None` as `{}`).
2. Locate the neighbor-generation loop in your A* code (where you normally check for physical walls).
3. Before adding a neighbor to the `open_list`, clear the KB facts.
4. Feed that tile's facts into the KB: `for fact in tile_facts.get(successor, []): self.kb.tell_fact(fact)`.
5. Run `self.kb.forward_chain()`.
6. If `'Retreat'` is deduced and exists in `self.kb.facts`, mark the tile as **Infeasible**. Skip it (do not add it to the open list), even if it is physically reachable.

Step 3.3: Run It in the Grid

1. In `sense_and_act()`, define sample facts and pass them to A*:

```
tile_facts = {
   (1, 0): ['TargetVisible', 'HasDust', 'BloodseekerMissing'],  # deduces Retreat: block
   (0, 1): ['TargetVisible', 'HasDust'],                        # SafeToEngage only: all
```

```
}
self.plan = self.astar_search(position, nearest_food, walls, grid_size, tile_facts=tile_f
```

2. Run `visual_grid_game.py`. The agent should never step onto `(1, 0)`.

Part 4: Self-Evaluation Test Cases

4.1 Automated Validation

Create a file named `test_logic.py`. Copy and run the following Python assert statements to verify your Forward Chaining engine works correctly mathematically before you attempt to integrate it into the visual grid.

```
 from logic_engine import KnowledgeBase

def test_forward_chaining():
    kb = KnowledgeBase()

    # Add Domain Rules
    kb.tell_rule(['TargetVisible', 'HasDust'], 'SafeToEngage')
    kb.tell_rule(['SafeToEngage', 'BloodseekerMissing'], 'Retreat')

    # Test Case 1: Safe Engagement
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.forward_chain()
    assert 'SafeToEngage' in kb.facts, "Test 1 Failed: Should deduce SafeToEngage"
    assert 'Retreat' not in kb.facts, "Test 1 Failed: Should NOT deduce Retreat"

    # Test Case 2: Unsafe Engagement (Bloodseeker Missing)
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.tell_fact('BloodseekerMissing')
    kb.forward_chain()
    assert 'Retreat' in kb.facts, "Test 2 Failed: Should deduce Retreat to override searc
```

```
    print("✅ All Logic Engine Test Cases Passed!")

if __name__ == "__main__":
    test_forward_chaining()
```

Part 5: Theory-to-Implementation Mapping

> Based on the code you just wrote and the lecture on Logical Reasoning, complete the following analytical questions to explain *how* your code implements the theory.

1. Reachability vs. Feasibility (Apply)

In Step 3.2, you modified the A* algorithm's neighbor-generation loop. How does your new code differentiate between checking if a node is "Reachable" (like a wall collision) versus checking if it is "Feasible"? Explain how the Knowledge Base enforces feasibility.

In our modified A* pathfinding architecture, spatial validity is strictly partitioned into two distinct evaluations: physical Reachability and logical Feasibility: 1. Reachability (Spatial & Kinematic Validity): A candidate node is physically reachable if it lies strictly within the grid boundaries (0 ≤ x < width and 0 ≤ y < height) and does not collide with static obstacles (next_pos not in walls_set). Reachability answers the purely geometric question: "Can the agent physically move into this tile without walking through an impassable wall?" 2. Feasibility (Logical Safety & Strategic Viability): A candidate node is logically feasible only if entering it satisfies all high-level domain constraints, safety invariants, and threat rules deduced from real-time perceptual intelligence. Feasibility answers the cognitive question: "Is it safe and strategically sound for the agent to enter this tile given current enemy and environmental conditions?" 3. Enforcement by the Knowledge Base: In Step 3.2, before adding any reachable neighbor to the A* priority queue (open_list), the algorithm feeds the candidate tile's percepts into the Knowledge Base via self.kb.tell_fact(fact). It then executes self.kb.forward_chain(). If the inference engine derives the conclusion 'Retreat' (e.g., TargetVisible ∧ HasDust ⇒ SafeToEngage, followed by SafeToEngage ∧ BloodseekerMissing ⇒ Retreat), the node is flagged as Infeasible. A* executes a 'continue' statement, pruning the node from expansion and preventing the agent from ever taking that step, regardless of how short or physically clear the path appears.

2. Declarative vs. Procedural Paradigms (Analyze)

Why did we build a `KnowledgeBase` class with a `forward_chain()` loop instead of simply writing

`if 'TargetVisible' in percepts and 'HasDust' in percepts:` directly inside the agent's movement code? What happens to the agent's code complexity if we add 50 new game rules?

We implemented a declarative KnowledgeBase with forward_chain() rather than hardcoding procedural conditional statements for several foundational AI architectural reasons: 1. Separation of Knowledge from Inference (Declarative vs. Procedural): In a procedural paradigm, domain rules are hard-coded into control flow logic using nested if-else blocks. This tightly couples "what the agent knows" with "how the agent decides," requiring changes to core movement algorithms whenever game rules change. Our declarative design decouples the domain knowledge (facts and Horn clauses) from the inference engine (Modus Ponens forward chaining). Domain rules can be added, updated, or removed declaratively without touching a single line of pathfinding or action-selection code. 2. Autonomous Multi-Step Deductive Chaining: Procedural if-statements can only easily check immediate, shallow combinations of raw percepts. They struggle with cascading inferences where fact A ∧ B entails C, and newly deduced C ∧ D entails E. Our forward_chain() engine iteratively evaluates rules until deductive closure (a fixed point) is achieved, automatically deriving all multi-tier entailments. 3. Code Complexity Scaling with 50 New Rules: - Procedural Scaling: Adding 50 interconnected rules in procedural code causes combinatorial explosion of nested branches (O(2^n) branching paths in the worst case). The code becomes unmaintainable "spaghetti logic" with extreme cyclomatic complexity and high risk of regression bugs. - Declarative Scaling: In our Knowledge Base, adding 50 rules simply requires 50 clean tell_rule() calls during setup. The agent's movement code cyclomatic complexity remains O(1), and forward chaining execution complexity remains polynomial O(R · P) where R is the number of rules and P is premise count. This guarantees modularity, maintainability, and clean scalability.

3. Modus Ponens & Horn Clauses (Understand)

The rule `['TargetVisible', 'HasDust'] ⇒ 'SafeToEngage'` is formally known as a Horn Clause. Briefly explain how your Python implementation of the `forward_chain()` `while` loop acts as a mathematical execution of the *Modus Ponens* inference rule.

A definite Horn Clause is an implication of the form $P_1 \land P_2 \land ... \land P_k \Rightarrow Q$, having a conjunction of positive antecedent premises and a single positive consequent literal. Generalized Modus Ponens states that whenever each individual premise P_i is known to be true, the conclusion Q must logically follow and can be asserted as a new truth: $[(P_1 \land ... \land P_k \Rightarrow Q) \land P_1 \land ... \land P_k] \vdash Q$ Our Python implementation mathematically mirrors this inference mechanism: 1. Rule Representation: Each Horn clause is stored as a tuple (premises, conclusion) where premises is [P_1, P_2, ...] and conclusion is Q. 2. Modus Ponens Antecedent Check: The statement 'if all(p in self.facts for p in premises):' tests the logical conjunction $P_1 \land P_2 \land ... \land P_k$. The Python all() generator evaluates whether every premise p belongs to self.facts (known truths) in O(1) average lookup time. 3. Conclusion Assertion: When the antecedent conjunction holds true and conclusion is not yet in self.facts, 'self.facts.add(conclusion)' executes the Modus Ponens step, adding the newly proven proposition Q to the knowledge base. 4. Deductive Closure: The outer 'while new_facts_added:' loop guarantees iterative completeness. By setting new_facts_added = True whenever a deduction occurs, the engine passes over all rules again, allowing newly derived conclusions (like 'SafeToEngage') to trigger higher-order rules (like 'Retreat'). When a complete pass yields no new facts (new_facts_added == False), a mathematical fixed point is reached, proving all consequences entailed by the KB have been fully deduced.

✅ Lab 07 Theory-to-Implementation Mapping completed!

FACULTY OF COMPUTING